

5 Cap Weighted Portfolio Plots → Run full factor pipeline → Two factor regressions FMB

```
[14]: start_date = "1999-10-01"
```

5.1 Filter all returns to match start date

```
[15]: asset_returns_trimmed = asset_returns.loc[start_date:]
caps_df_trimmed = caps_df.loc[start_date:]
factor_returns_trimmed = factor_returns.loc[start_date:]
betas_ebc_trimmed = betas_ebc.loc[start_date:]
betas_cw_ebc_trimmed = betas_cw_ebc.loc[start_date:]
```

```
[45]: n_q1, n_q2 = 1,3
```

```
[46]: cw_results = fmb_pipe.run_full_factor_pipeline(
    beta_EBC_df = betas_ebc_trimmed,
    beta_CW_EBC_df=betas_cw_ebc_trimmed,
    returns_df=asset_returns_trimmed,
    caps_df = caps_df_trimmed,
    EBC_returns_df=factor_returns_trimmed["EBC"],
    Cap_EBC_returns_df=factor_returns_trimmed["CW-EBC"],
    cap_weights=True,
    n_q1 = n_q1,
    n_q2 = n_q2,
    frequency = 'monthly',
    formation_frequency='quarterly',
    min_assets_per_cell=3,
    f1_name="EBC",
    f2_name="CW-EBC"
)

cw_results["mean_grid"]*12
```

Q1 buckets: 0% | 0/1 [00:00<?, ?it/s]

```
[46]:          1          2          3
1  0.091589  0.092197  0.109428
```

```
[48]: rename_map = {
    "Q1_Q1": "High Pref",
    "Q1_Q2": "Mid Pref",
    "Q1_Q3": "Low Pref"
}
# Save pre-rename copy for true geometric mean computation (needs Q1_Q1 style,
# ↪ column names)
cw_portrets_raw = cw_results['portrets_wide'].copy()
```

```

cw_results['portrets_wide'] = cw_results['portrets_wide'].
↳rename(columns=rename_map)

```

```

[18]: rename_map = {
    "Q1_Q1": "EBC_High_Pref_High",
    "Q1_Q2": "EBC_High_Pref_Mid",
    "Q1_Q3": "EBC_High_Pref_Low",
    "Q2_Q1": "EBC_Mid_Pref_High",
    "Q2_Q2": "EBC_Mid_Pref_Mid",
    "Q2_Q3": "EBC_Mid_Pref_Low",
    "Q3_Q1": "EBC_Low_Pref_High",
    "Q3_Q2": "EBC_Low_Pref_Mid",
    "Q3_Q3": "EBC_Low_Pref_Low",
}

# Save pre-rename copy for true geometric mean computation (needs Q1_Q1 style,
↳column names)
cw_portrets_raw = cw_results['portrets_wide'].copy()
cw_results['portrets_wide'] = cw_results['portrets_wide'].
↳rename(columns=rename_map)

```

```

[67]: full_merged_df.index = full_merged_df.index.to_timestamp(how="start")

```

```

[68]: cw_complete_df = full_merged_df.join(
    cw_results['portrets_wide'],
    how="inner"
)
selected_columns = ['CW',
    'EBC_High_Pref_High', 'EBC_High_Pref_Mid',
↳'EBC_High_Pref_Low',
    'EBC_Mid_Pref_High', 'EBC_Mid_Pref_Mid', 'EBC_Mid_Pref_Low',
    'EBC_Low_Pref_High', 'EBC_Low_Pref_Mid', 'EBC_Low_Pref_Low']

```

```

[69]: cw_complete_df = full_merged_df.join(
    cw_results['portrets_wide'],
    how="inner"
)

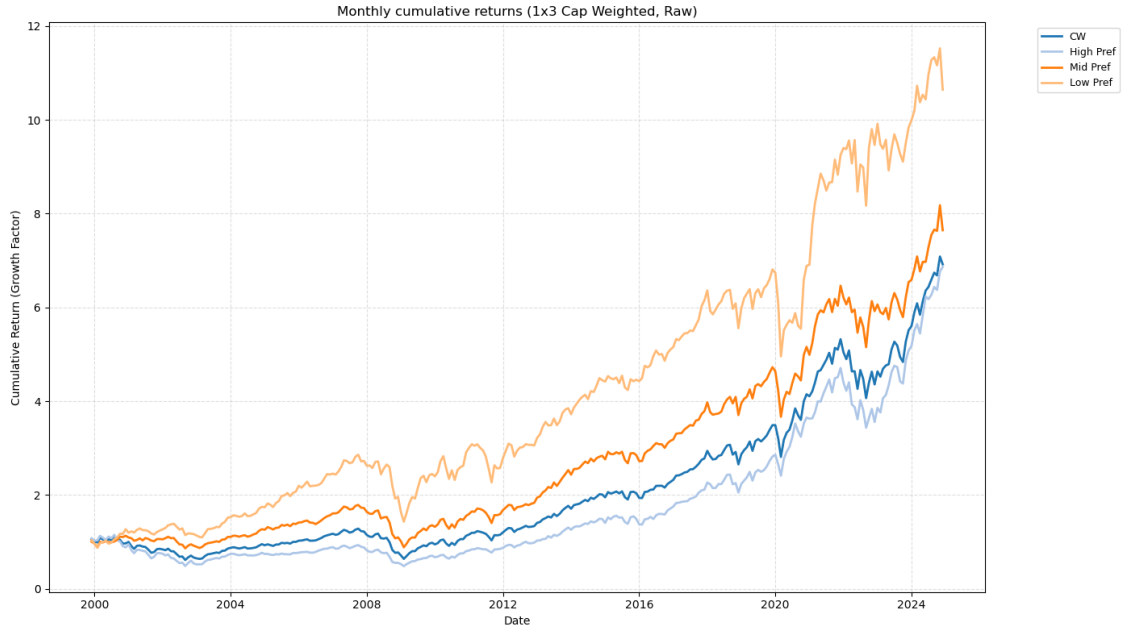
selected_columns = ["CW", "High Pref", "Mid Pref", "Low Pref"]

```

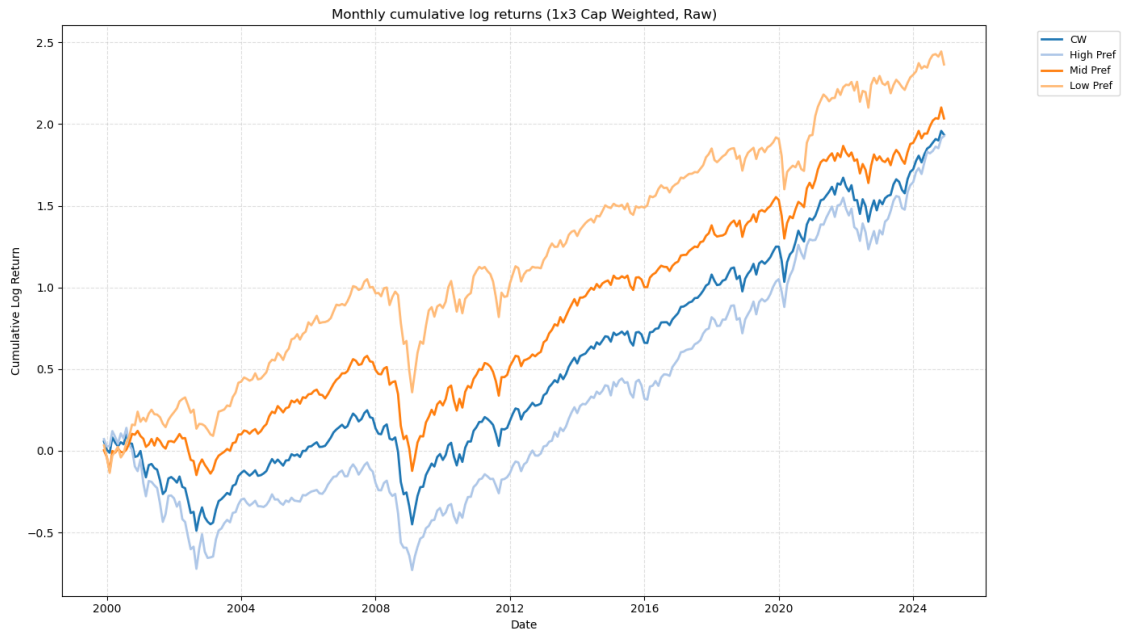
```

[65]: fmb_plots.plot_cum_returns(cw_complete_df[selected_columns], title=f"Monthly
↳cumulative returns ({n_q1}x{n_q2} Cap Weighted, Raw)", log=False)

```



```
[66]: fmb_plots.plot_cum_returns(cw_complete_df[selected_columns], title = f"Monthly_
      ↪cumulative log returns ({n_q1}x{n_q2} Cap Weighted, Raw)")
```

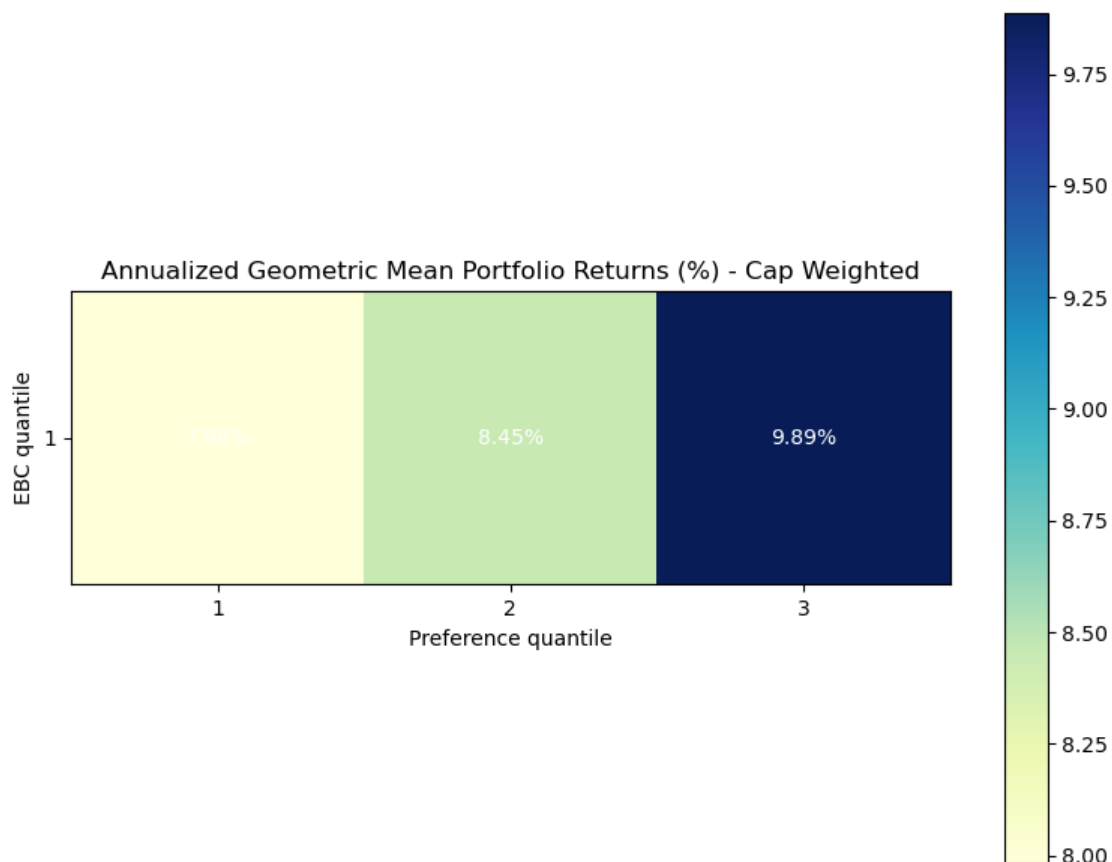


```
[70]: period = start_date[:4] + '_2024'
      save_dir = DATA_PROCESSED_DIR / 'monthly'
```

```
[71]: monthly_cw_quantile_returns = cw_complete_df[selected_columns]
monthly_cw_quantile_returns.to_csv(save_dir /
    ↪ f"monthly_{period}_cw_quantile_returns_{n_q1}x{n_q2}.csv")

[24]: # Arithmetic mean grid (annualized as mean × 12)
#fmb_plots.plot_mean_grid_new(cw_results["mean_grid"], annualize=True, cw=True,
    ↪ geometric=False)

[72]: # True geometric mean grid (annualized as (1+geo_monthly)^12 - 1)
cw_actual_mean_grid = fmb_plots.plot_mean_grid_new(
    cw_results["mean_grid"], annualize=True, cw=True, geometric=True,
    portrets_wide=cw_portrets_raw
)
```



```
[55]: # Benchmark: annualized geometric mean returns over the analysis period
cw_geo = np.exp(1/12 * np.log1p(cw_complete_df['CW']).mean())
cw_ann = (1 + cw_geo) ** 12 - 1
print(f"Annualized Geometric Mean Return ({start_date[:4]}-2024)")
print(f"  CW (ours) : {cw_ann:.2%}")
```

```

try:
    spy_geo = np.exp(1/12 * np.log1p(spy_ret.loc[start_date:]).mean())
    spy_ann = (1 + spy_geo) ** 12 - 1
    print(f"    SPY          : {spy_ann:.2%}")
except Exception:
    print("    SPY          : (run ETF cell first)")

```

Annualized Geometric Mean Return (1999–2024)

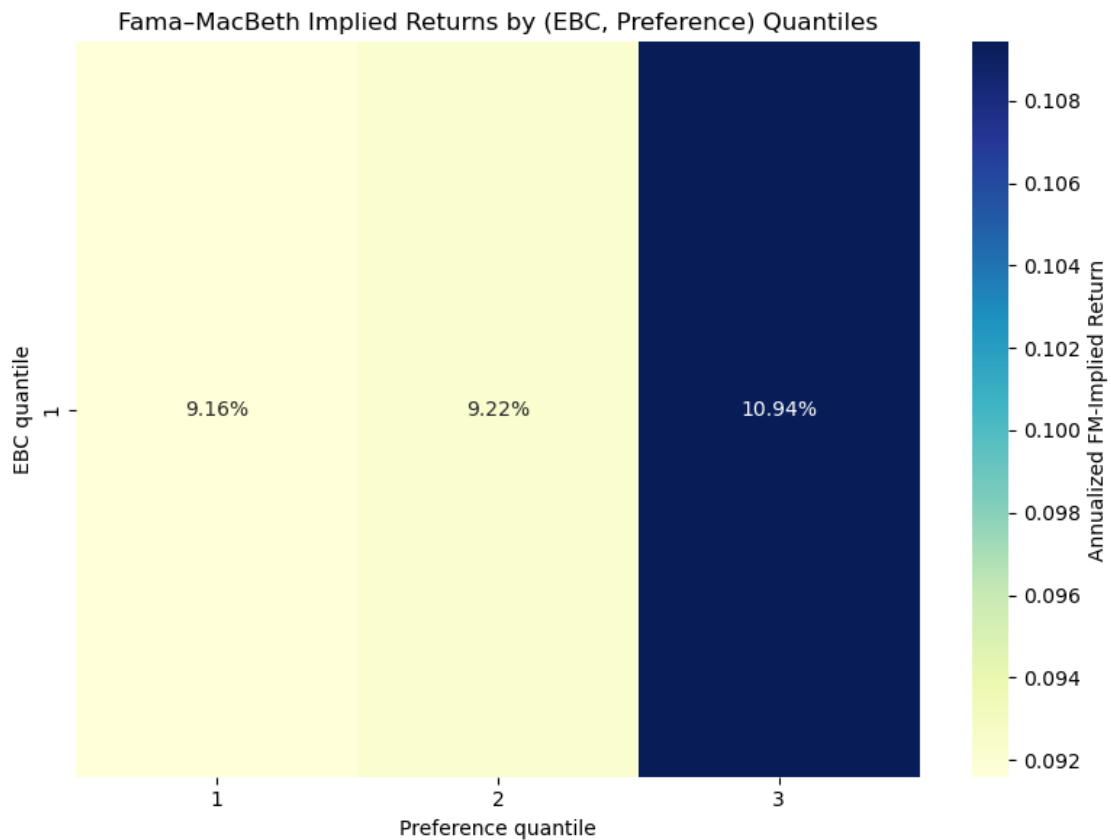
CW (ours) : 8.02%

SPY : 8.11%

```

[73]: cw_implied_mean_grid = fmb_plots.
      ↪ plot_expected_return_grid(cw_results["pricing"], cw_results["fm_table"],
      ↪ n_q1, n_q2)

```



6 Equal Weighted Portfolio Plots → Run full factor pipeline → Two factor regressions FMB

```
[74]: ew_results = fmb_pipe.run_full_factor_pipeline(
    beta_EBC_df = betas_ebc_trimmed,
    beta_CW_EBC_df=betas_cw_ebc_trimmed,
    returns_df=asset_returns_trimmed,
    caps_df = caps_df_trimmed,
    EBC_returns_df=factor_returns_trimmed["EBC"],
    Cap_EBC_returns_df=factor_returns_trimmed["CW-EBC"],
    cap_weights=False,
    n_q1 = n_q1,
    n_q2 = n_q2,
    frequency = 'monthly',
    formation_frequency='quarterly',
    min_assets_per_cell=3,
    f1_name="EBC",
    f2_name="CW-EBC"
)

ew_results["mean_grid"]*12
```

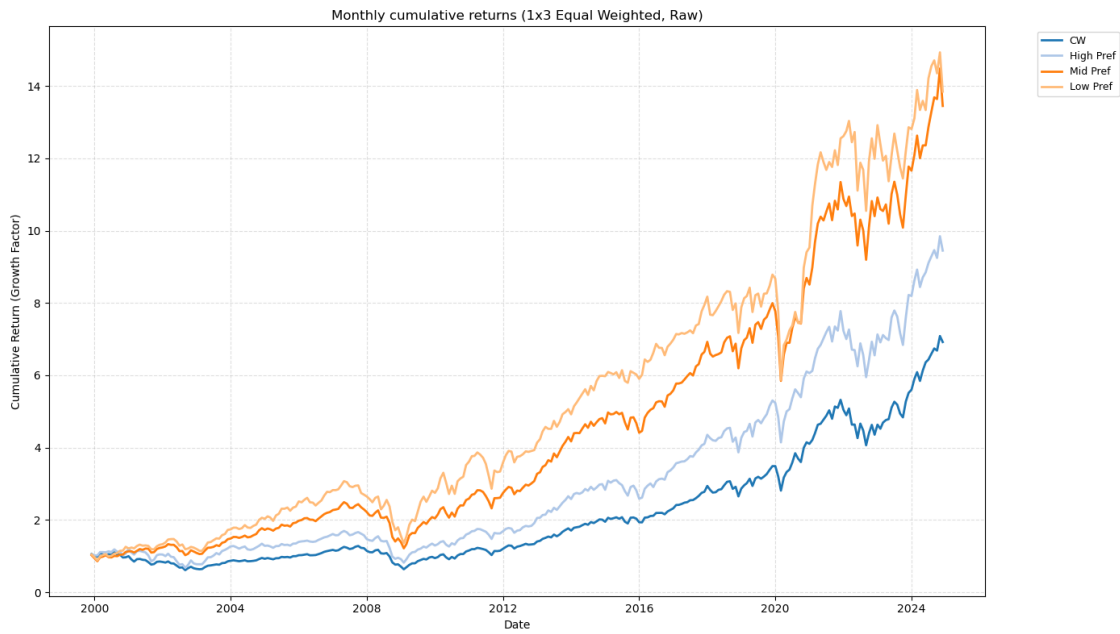
Q1 buckets: 0% | 0/1 [00:00<?, ?it/s]

```
[74]:      1      2      3
1  0.106474  0.117427  0.123457
```

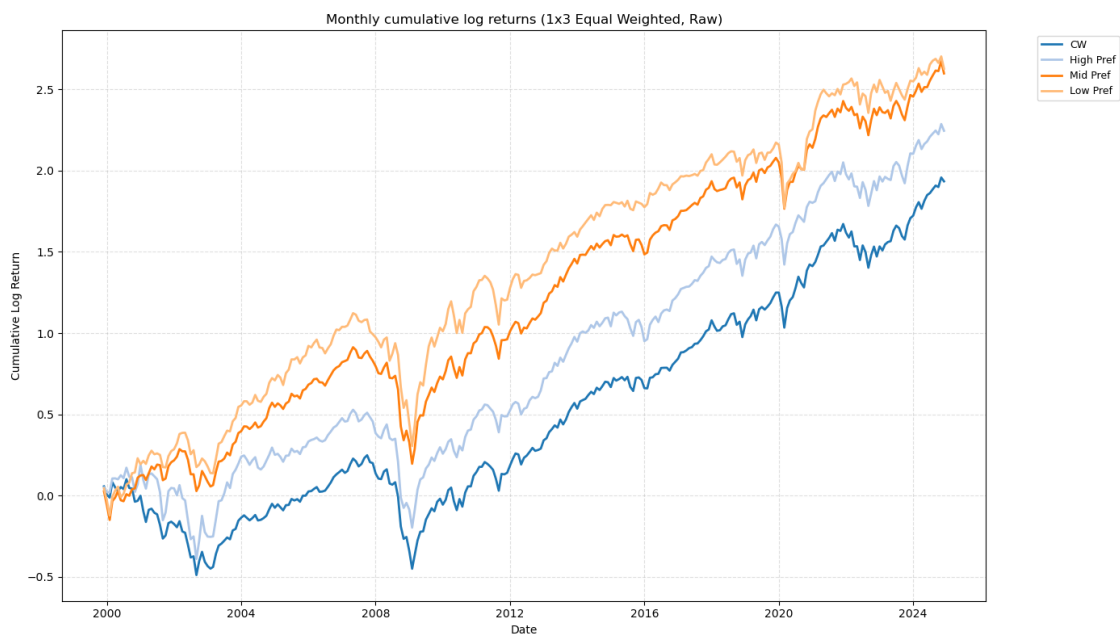
```
[76]: # Save pre-rename copy for true geometric mean computation (needs Q1_Q1 style,
    ↪column names)
ew_portrets_raw = ew_results['portrets_wide'].copy()
ew_results['portrets_wide'] = ew_results['portrets_wide'].
    ↪rename(columns=rename_map)
```

```
[77]: ew_complete_df = full_merged_df.join(
    ew_results['portrets_wide'],
    how="inner"
)

fmb_plots.plot_cum_returns(ew_complete_df[selected_columns],
    title=f"Monthly cumulative returns,
    ↪({n_q1}x{n_q2} Equal Weighted, Raw)", log=False)
```



```
[78]: fmb_plots.plot_cum_returns(ew_complete_df[selected_columns],
                                title=f"Monthly cumulative log returns_
                                ↳({n_q1}x{n_q2} Equal Weighted, Raw)",log = True)
```

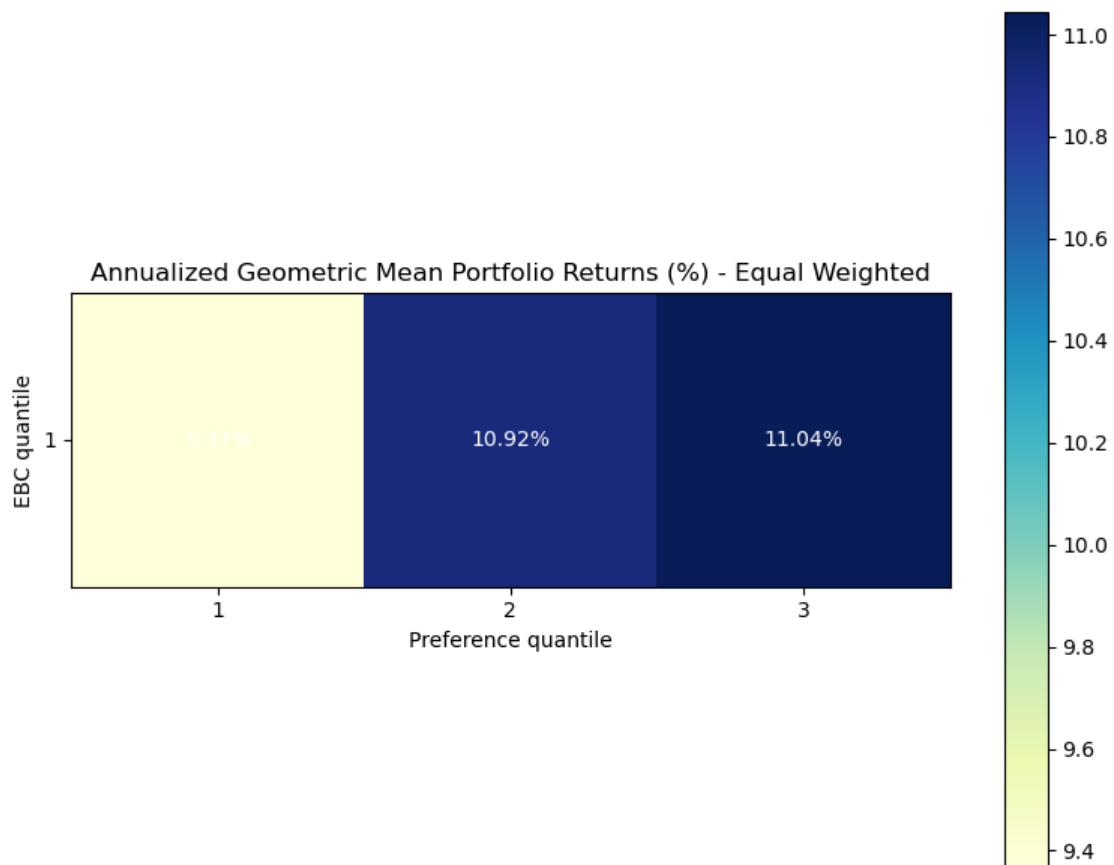


```
[61]: monthly_ew_quantile_returns = ew_complete_df[selected_columns]
```

```
[79]: monthly_ew_quantile_returns.to_csv(save_dir /
    ↪ f"monthly_{period}_ew_quantile_returns_{n_q1}x{n_q2}.csv")

[80]: # Arithmetic mean grid (annualized as mean × 12)
    #fmb_plots.plot_mean_grid_new(ew_results["mean_grid"], annualize=True,
    ↪ frequency='monthly', cw=False, geometric=False)

[81]: # True geometric mean grid (annualized as (1+geo_monthly)^12 - 1)
    ew_actual_mean_grid = fmb_plots.plot_mean_grid_new(
        ew_results["mean_grid"], annualize=True, frequency='monthly', cw=False,
    ↪ geometric=True,
        portrets_wide=ew_portrets_raw
    )
```



```
[82]: # Benchmark: annualized geometric mean returns over the analysis period
    cw_geo = np.exp(1/np.log1p(cw_complete_df['CW']).mean())
    cw_ann = (1 + cw_geo) ** 12 - 1
    print(f"Annualized Geometric Mean Return ({start_date[:4]}-2024)")
    print(f"  CW (ours) : {cw_ann:.2%}")
```

```

try:
    spy_geo = np.exp(1/12 * np.log1p(spy_ret.loc[start_date:]).mean())
    spy_ann = (1 + spy_geo) ** 12 - 1
    print(f"    SPY          : {spy_ann:.2%}")
except Exception:
    print("    SPY          : (run ETF cell first)")

```

Annualized Geometric Mean Return (1999–2024)

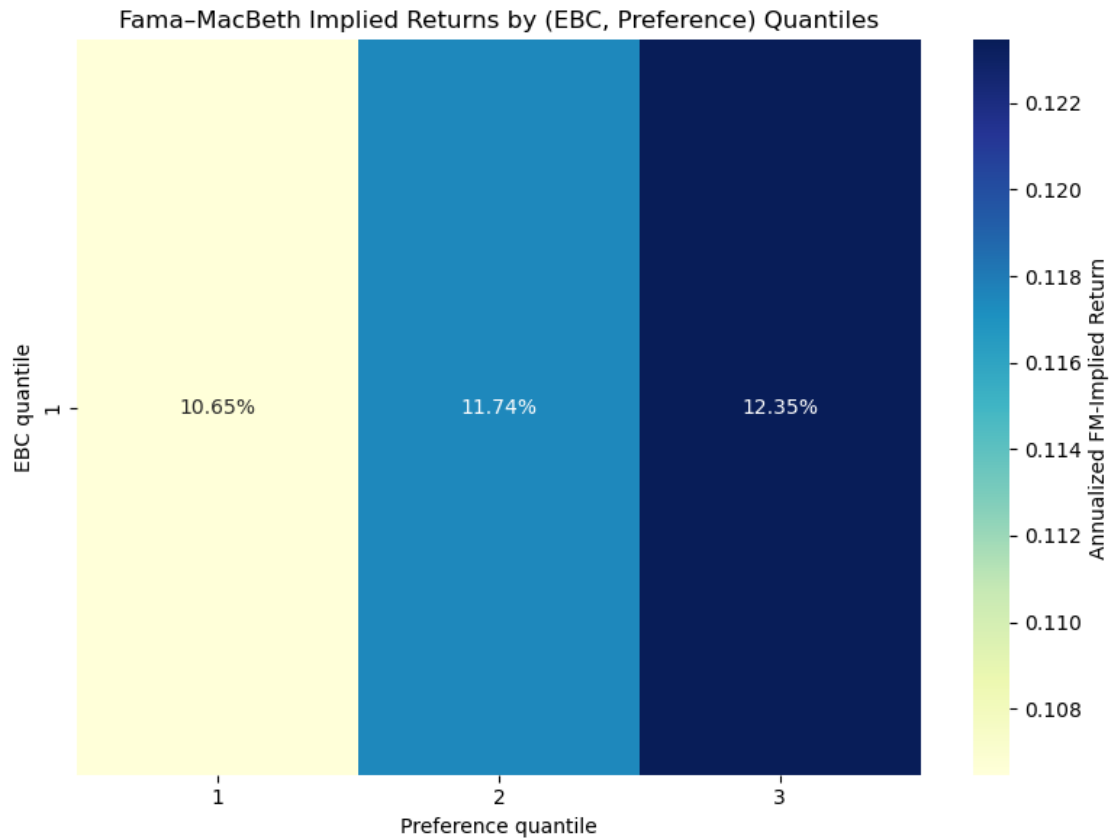
CW (ours) : 8.02%

SPY : 8.11%

```

[83]: ew_implied_mean_grid = fmb_plots.
      ↪ plot_expected_return_grid(ew_results["pricing"], ew_results["fm_table"],
      ↪ n_q1, n_q2)

```



7 Fitted GARCH

```

[38]: from arch import arch_model

```